

Introduction au Python et premiers petits calculs



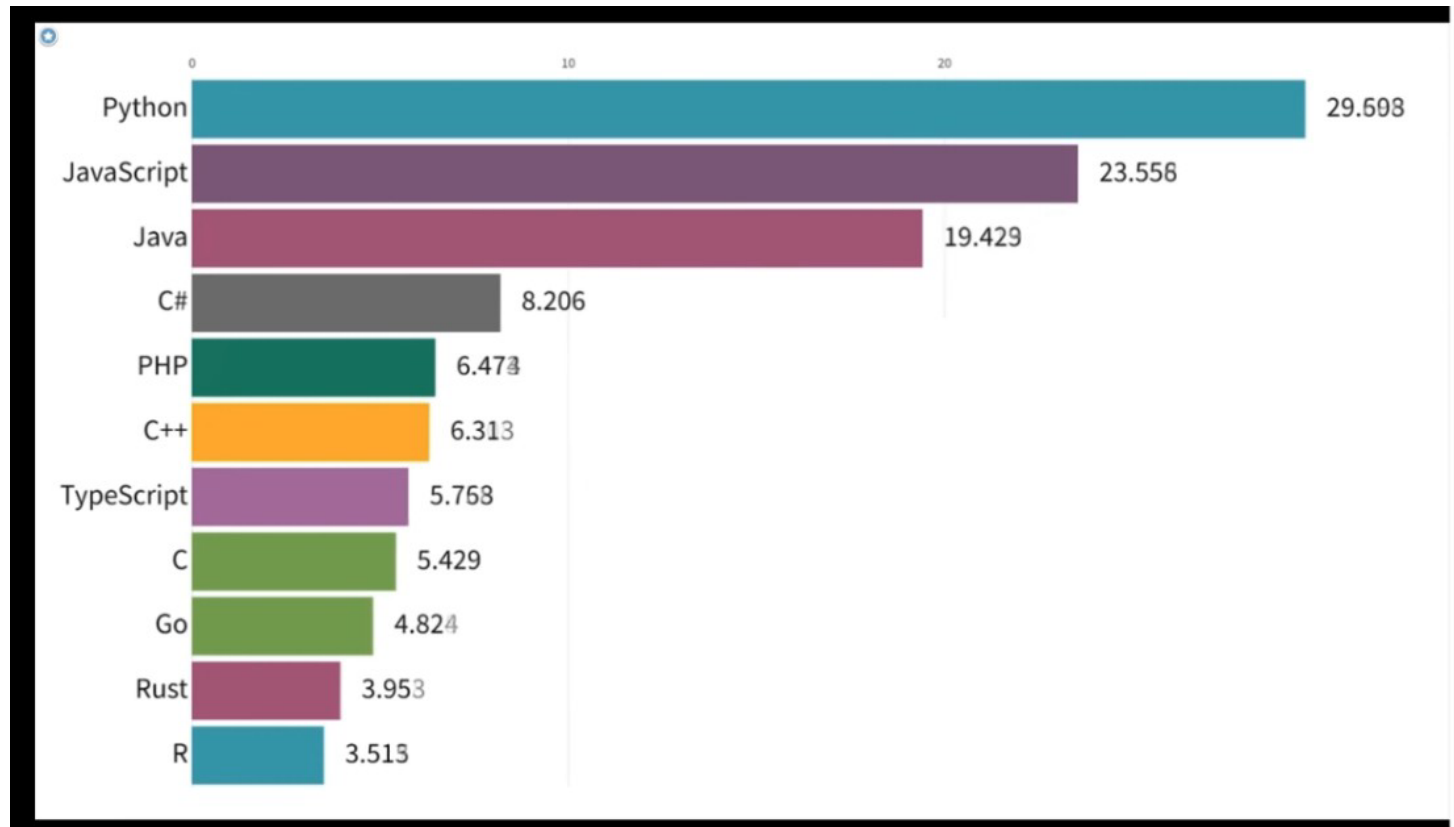
BTS CIEL-IR

Cours : Maths -Info

Activités : R2 ; D5

Compétences : C08 ; C09 ; C10

Pourquoi utiliser Python et pourquoi est-il le langage à apprendre en 2024 ?



[Prez_Maths_Info_01.mp4]

I. Petite histoire de Python.

Le Python est un langage de programmation **interprété**, **multi-paradigme** et **multiplateformes**. Il favorise la programmation **impérative structurée**, **fonctionnelle** et **orientée objet**.

Le langage Python est placé sous une **licence libre**.



C'est un langage de programmation qui peut s'utiliser dans de **nombreux contextes** et s'**adapter à tout type d'utilisation** grâce à des modules (*bibliothèques*) spécialisées.



Ses champs d'implications sont :

- les **scripts d'automatisation** des tâches simples mais fastidieuses ;
- le **développement de prototype** lorsqu'on a besoin d'une application fonctionnelle avant de l'optimiser avec un langage de plus bas niveau ;
- le **quantique** ;
- l'**intelligence artificielle** ;
- le traitement des **Big Data** ;
- le **Machine learning**.

Extrait Wikipédia

II. Quelques caractéristiques



Syntaxe

Python a été conçu pour être un **langage lisible** qui vise à être visuellement **épuré**. Les blocs sont identifiés par l'**indentation** (et non par d'accolades). L'**augmentation** de l'indentation marque le **début** d'un bloc, et sa **réduction** en marque la **fin**.

Fonction **factorielle** en C

```
int factorielle(int n) {  
    if (n < 2) {  
        return 1;  
    } else {  
        return n *  
        factorielle(n - 1);  
    }  
}
```

Fonction **factorielle** en Python

```
def factorielle(n):  
    if n < 2:  
        return 1  
    else:  
        return n *  
        factorielle(n - 1)
```

Programmation fonctionnelle

Python permet de programmer dans un style **fonctionnel**. Il dispose également des compréhensions de **listes**, et plus généralement les compréhensions peuvent **produire des générateurs**, des **dictionnaires** ou des **ensembles**.

Ainsi, pour construire la liste des carrés des entiers naturels plus petits que 10, on peut utiliser l'expression :

```
liste = [x**2 for x in range(10)]  
# liste = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```


Programmation objet

(que vous appréhenderez avec Monsieur To l'année prochaine.)

Tous les types de base, les fonctions, les **instances de classes** (les objets « classiques » des langages C++ et Java) et les **classes elles-mêmes** (qui sont des instances de méta-classes) **sont des objets**. Une classe se définit avec le mot-clé `class`.

```
class Personne:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom
    def presenter(self):
        return self.nom + " " + self.prenom

class Etudiant(Personne):
    def __init__(self, niveau, nom, prenom):
        Personne.__init__(self, nom, prenom)
        self.niveau = niveau
    def presenter(self):
        return self.niveau + " " + Personne.presenter(self)

e = Etudiant("Licence INFO", "Dupontel", "Albert")
assert e.nom == "Dupontel"
```

Bibliothèque standard

Python possède une **grande bibliothèque standard**, fournissant des outils convenant à de nombreuses tâches diverses. Le nombre de **modules de la bibliothèque standard** peut être augmenté avec des modules spécifiques écrits en C ou en Python.



La communauté Python

Forte et impliquée, la communauté Python se retrouve notamment sur le forum Usenet anglophone comp.lang.python, ainsi que sur des forums, dans des mailing lists, et sur réseaux sociaux(Reddit ou Discord). **GitHub facilite la collaboration ouverte et la contribution au code source de Python.** Les utilisateurs et développeurs de bibliothèques tierces peuvent y déposer leurs contributions.



III Quelques règles de base pour la programmation en Python

Utiliser des espaces blancs entre les opérateurs et ne pas les coller les éléments du calcul :

a = f(1, 2) + g(3, 4).

Utiliser la convention pour les classes et les fonctions :
English, minuscule, underscore si séparateur et enfin CamelCase :

lower_case_with_underscores

Ne jamais utiliser d'accentuations.

Respecter **l'indentation**.

pour chaque base dans séquence:

```
← indentation  afficher(base)
                taille <- taille + 1
                afficher(taille)
```

↖ bloc d'instructions